

Tópicos Avançados em IA Generativa
Docente: Prof. Dr. Anderson da Silva Soares
Discente: Sandro Silva Moreira

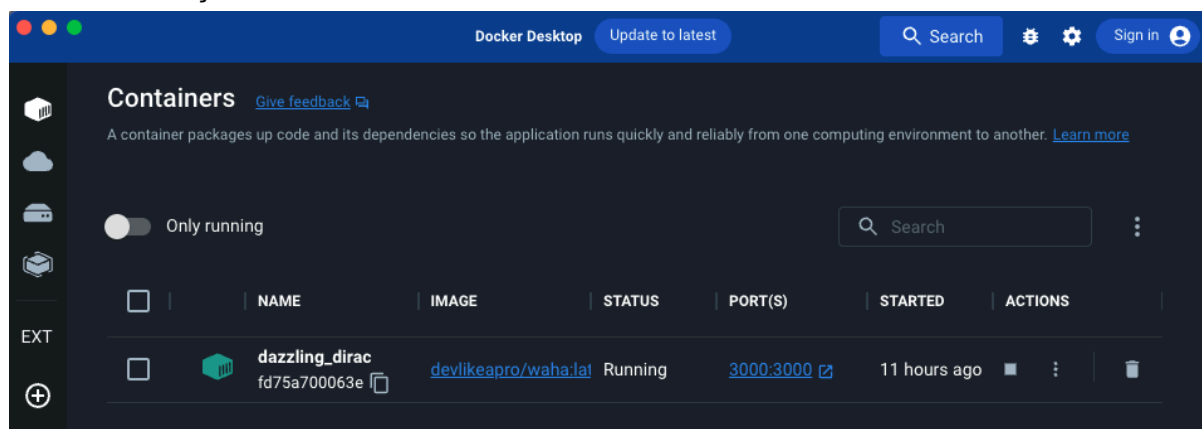
RELATÓRIO DE IMPLEMENTAÇÃO DE SOFTWARE

Requisitos da Atividade:

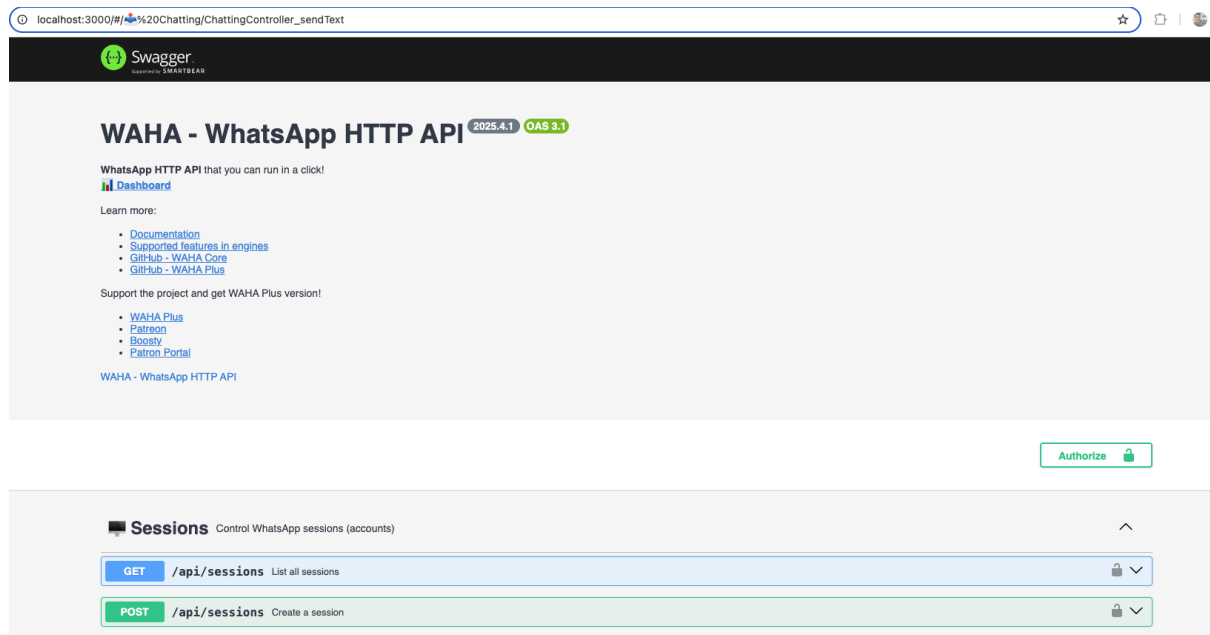
Construir um servidor MCP para envio de mensagens no WhatsApp utilizando o WAHA - O servidor deverá ter a tool: `send_message` que recebe como parâmetros o número no formato internacional (+5511..) e a mensagem a ser enviada. A tool deverá se comunicar com um servidor local de WaHa onde deve fazer uma chamada post na rota de envio de mensagens. Para instanciar o servidor de WaHa você deve seguir os passos do video: <https://www.youtube.com/watch?v=RFerMyAUPRg> <https://waha.devlike.pro/docs/overview/quick-start/> Com o servidor autenticado, a tool deve fazer uma chamada POST na rota `/api/sendText`, passando como parâmetros o número recebido e a mensagem. Exemplo de chamada em python: `import requests url = "http://localhost:3000/api/sendText" headers = { "Accept": "application/json", "Content-Type": "application/json" } data = { "chatId": "123123@c.us", "text": "Hi there!", "session": "default" } response = requests.post(url, json=data, headers=headers) print(response.json())` Implemente também um resource no servidor que tenha 3 números de telefone e nome já pré carregados para que o client de MCP utilize ele ao ser chamado com: "Envie uma mensagem de bom dia para o João". Entregável: Github ou outro repositório relatório básico de uso com print de funcionalidade de um servidor funcional com 1 tool (`send_message`) e 1 resource (contatos). Referências: <https://waha.devlike.pro/docs/overview/quick-start/> https://www.youtube.com/watch?v=RFerMyAUPRg&ab_channel=devlikeapro <https://modelcontextprotocol.io/quickstart/server>

Implementação:

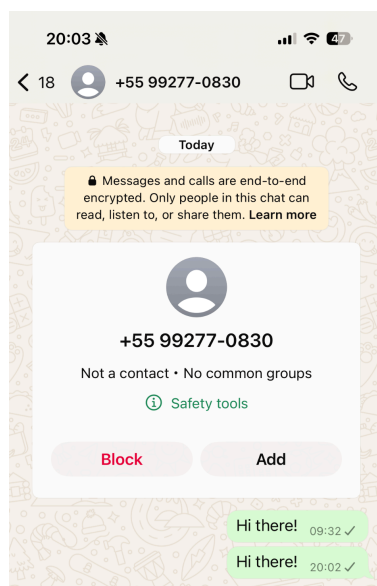
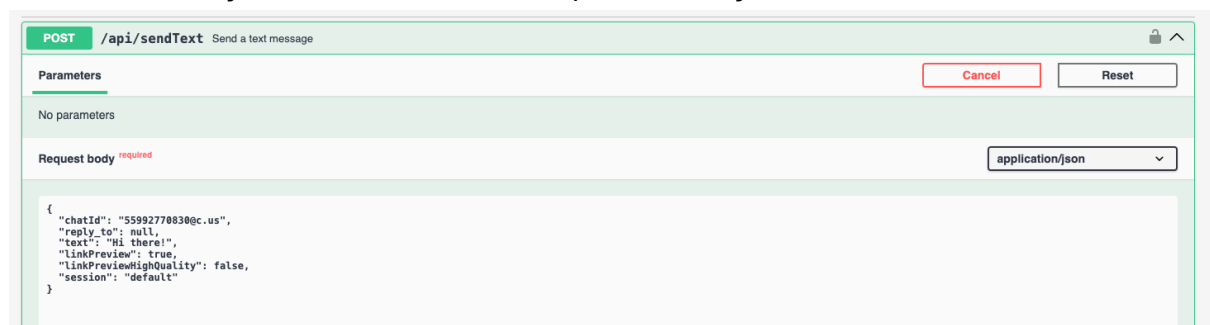
Passo 1 - Criação de Docker Container com WAHA



Passo 2 - Autenticação da API através QRCode do WhatsApp via Swagger



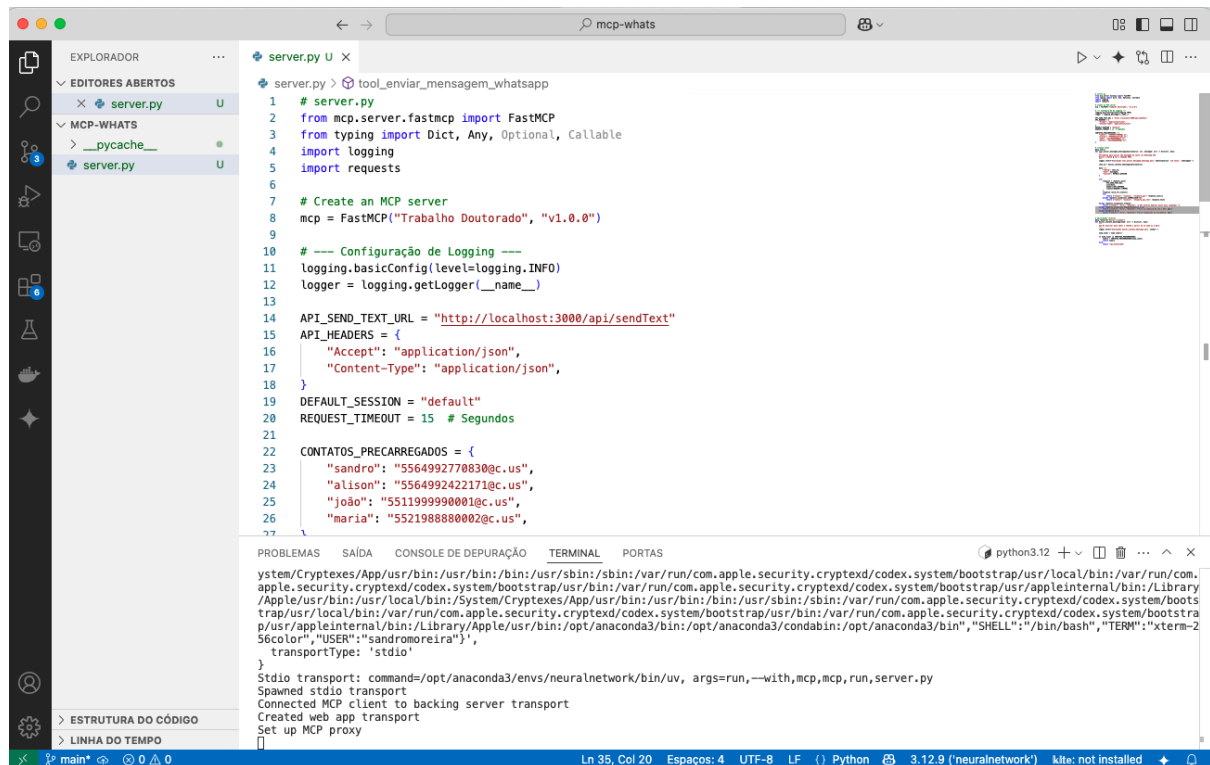
Passo 3 - Execução do EndPoint sendText para verificação de funcionamento



(Mensagem recebida no Aplicativo WhatsApp)

Passo 4 - Desenvolvimento do Servidor MCP (codificado em Python 3.12.9)

```
# mkdir mc-whats
# cd mcp-whats
# pip install fastapi uvicorn requests
# git init
# code .
```



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'mcp-whats' with files 'server.py' and '__pycache__'. The code editor displays the content of 'server.py', which is a Python script for an MCP server. The terminal shows the output of running the script, indicating that the server is running successfully.

```
server.py > tool_enviar_mensagem_whatsapp
1 # server.py
2 from mcp.server.fastmcp import FastMCP
3 from typing import Dict, Any, Optional, Callable
4 import logging
5 import requests
6
7 # Create an MCP server
8 mcp = FastMCP("Trabalho Doutorado", "v1.0.0")
9
10 # --- Configuração de Logging ---
11 logging.basicConfig(level=logging.INFO)
12 logger = logging.getLogger(__name__)
13
14 API_SEND_TEXT_URL = "http://localhost:3000/api/sendText"
15 API_HEADERS = {
16     "Accept": "application/json",
17     "Content-Type": "application/json",
18 }
19 DEFAULT_SESSION = "default"
20 REQUEST_TIMEOUT = 15 # Segundos
21
22 CONTATOS_PRECARREGADOS = {
23     "sandro": "5564992770830@c.us",
24     "alison": "5564992422171@c.us",
25     "joão": "551199990001@c.us",
26     "maria": "552198880002@c.us",
27 }
```

python3.12 + ... x
system/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/opt/anaconda3/bin:/opt/anaconda3/condabin:/opt/anaconda3/bin, "SHELL": "/bin/bash", "TERM": "xterm-256color", "USER": "sandromoreira"},
{transportType: 'stdio'
}
Stdio transport: command=/opt/anaconda3/envs/neuralnetwork/bin/uv, args=run, --with,mcp,mcp,run,server.py
Spawned stdio transport
Connected MCP client to backing server transport
Created web app transport
Set up MCP proxy

Arquivo server.py - Importação de pacotes

```
from mcp.server.fastmcp import FastMCP
from typing import Dict, Any, Optional, Callable
import logging
import requests
```

Create an MCP server

```
mcp = FastMCP("Trabalho Doutorado", "v1.0.0")
```

--- Configuração de Logging ---

```
logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
```

--- URI do WAHA ---

```
API_SEND_TEXT_URL = "http://localhost:3000/api/sendText"
```

```
API_HEADERS = {
```

```

        "Accept": "application/json",
        "Content-Type": "application/json",
    }

    DEFAULT_SESSION = "default"
    REQUEST_TIMEOUT = 15 # Segundos

# --- Lista de contatos pré-carregados ---
CONTATOS_PRECARREGADOS = {
    "sandro": "5564992770830@c.us",
    "alison": "5564992422171@c.us",
    "joão": "5511999990001@c.us",
    "maria": "5521988880002@c.us",
}

# Criando tool que envia a mensagem via whatsapp
@mcp.tool()
def tool_enviar_mensagem_whatsapp(destinatario: str, mensagem: str) -> Dict[str, Any]:
    """
    Ferramenta para enviar uma mensagem de texto via WhatsApp API.
    Busca o chatId e faz a chamada POST.
    """
    logger.info(f"Executando tool_enviar_mensagem_whatsapp para '{destinatario}' com
    texto: '{mensagem}'")

    chat_id = buscar_contato_whatsapp(destinatario)

    data = {
        "chatId": chat_id,
        "text": mensagem,
        "session": DEFAULT_SESSION
    }

    try:
        response = requests.post(
            API_SEND_TEXT_URL,
            json=data,
            headers=API_HEADERS,
            timeout=REQUEST_TIMEOUT
        )
        response.raise_for_status()
    except requests.exceptions.JSONDecodeError:
        return {"status": "sucesso", "resposta_api": response.json()}
    except requests.exceptions.JSONDecodeError:
        return {"status": "sucesso", "resposta_api_raw": response.text}

```

```

except requests.exceptions.Timeout:
    return {"status": "erro", "detalhe": "A API externa demorou muito para responder."}
except requests.exceptions.RequestException as e:
    return {"status": "erro", "detalhe": f"Erro na comunicação com a API: {e}"}
except Exception as e:
    return {"status": "erro", "detalhe": f"Erro inesperado na ferramenta: {e}"}

# adicionando recursos
@mcp.resource("buscar://{nome}")
def buscar_contato_whatsapp(nome: str) -> Dict[str, Any]:
    """
    Função auxiliar para obter o chatId a partir de um nome ou número.
    """
    logger.info(f"Executando buscar_contato_whatsapp para '{nome}'")

    nome_lower = nome.lower()

    if nome_lower in CONTATOS_PRECARREGADOS:
        numero = CONTATOS_PRECARREGADOS[nome_lower]
        return numero
    else:
        return "nao_encontrado"

```

Voltando ao terminal

```

(neuralnetwork) MacBook-Pro-de-Sandro:mcp-whats sandromoreira$ uvicorn mcp_server:app --reload --port 8000
INFO:      Will watch for changes in these directories: ['/Users/sandromoreira/agentes/mcp-whats']
INFO:      Uvicorn running on                               (Press CTRL+C to quit)
INFO:      Started reloader process [14475] using StatReload

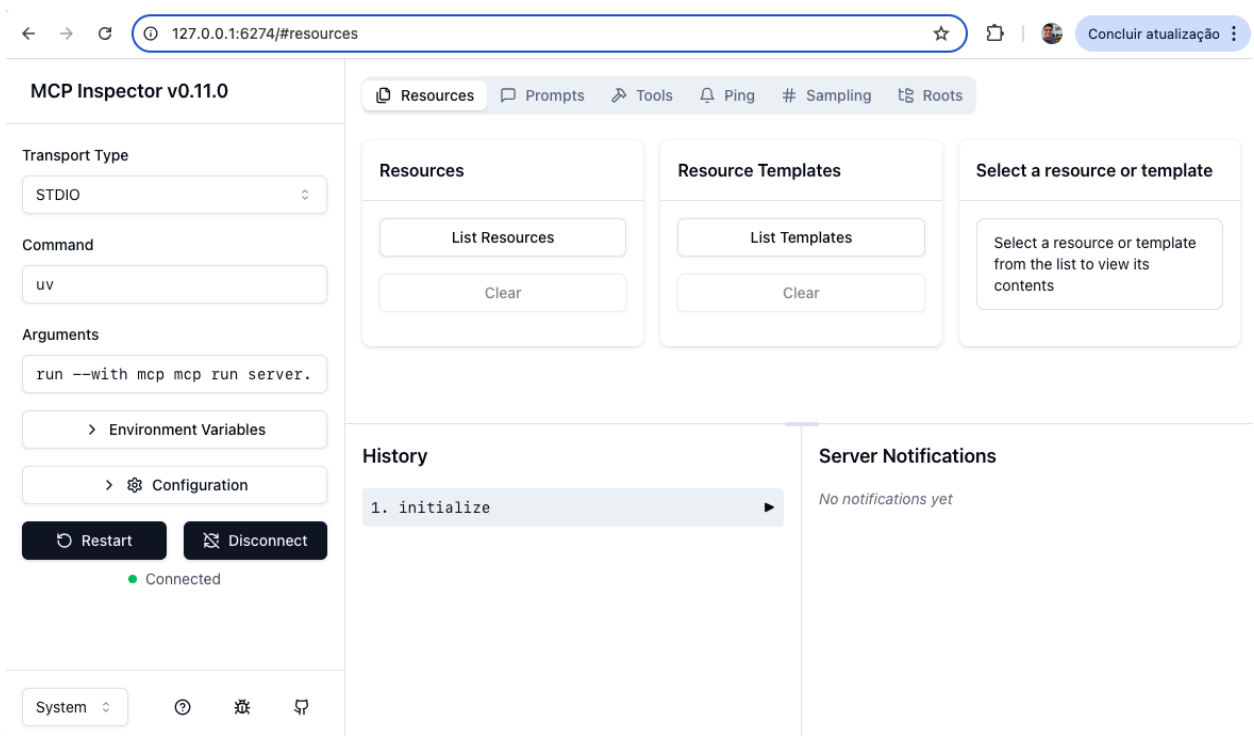
```

Passo 5 - Validando MCP Server através do MCP Inspector - Iniciando o Inspector

PROBLEMAS SAÍDA CONSOLE DE DEPURAÇÃO **TERMINAL** PORTAS

```
o (neuralnetwork) (base) MacBook-Pro-de-Sandro:mcp-whats sandromoreira$ mcp dev server.py
Starting MCP inspector...
Proxy server listening on port 6277
MCP Inspector is up and running at http://127.0.0.1:6274 🚀
New connection
Query parameters: [Object: null prototype] {
  command: 'uv',
  args: 'run --with mcp mcp run server.py',
  env: '{"HOME":"/Users/sandromoreira","LOGNAME":"sandromoreira","PATH":"/Users/sandromoreira/.npm/_npx/5a9d879542beca3a/node_modules/.bin:/Users/sandromoreira/node_modules/.bin:/Users/node_modules/.bin:/node_modules/.bin:/usr/local/lib/node_modules/npm/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/opt/anaconda3/envs/neuralnetwork/bin:/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/local/bin:/var/run/com.apple.security.cryptexd/codex.system/bootstrap/usr/bin:/usr/local/bin:/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:/var/run/com.apple.security.cryptexd/codex.system/apple.security.cryptexd/codex.system/bootstrap/usr/appleinternal/bin:/Library/Apple/usr/bin:/opt/anaconda3/bin:/opt/anaconda3/transportType: 'stdio'
}
Stdio transport: command=/opt/anaconda3/envs/neuralnetwork/bin/uv, args=run,--with,mcp,mcp,run,server.py
Spawned stdio transport
Connected MCP client to backing server transport
Created web app transport
Set up MCP proxy
□
```

Acessando o MCP Inspector (<https://127.0.0.1:6274>)



Na aba “Tools”, clicando em “List Tools” é possível ver que a ferramenta **tool_enviar_mensagem_whatsapp** está disponível

Tools

List Tools

Clear

tool_enviar_mensagem_whatsapp

Ferramenta para enviar uma mensagem de texto via WhatsApp API. Busca o chatId e faz a chamada POST.

tool_enviar_mensagem_whatsapp

Ferramenta para enviar uma mensagem de texto via WhatsApp API. Busca o chatId e faz a chamada POST.

destinatario

mensagem

Run Tool

Na aba “Resources” é possível observar o template recurso **buscar_contato_whatsapp**

Resources

List Resources

Clear

Resource Templates

List Templates

Clear

buscar_contato_whatsapp

buscar_contato_whatsapp

Função auxiliar para obter o chatId a partir de um nome ou número.

nome

Enter nome

Read Resource

É possível testar a busca de um nome:

buscar://sandro

Refresh

{

contents: [

0: {

uri: "buscar://sandro"

mimeType: "text/plain"

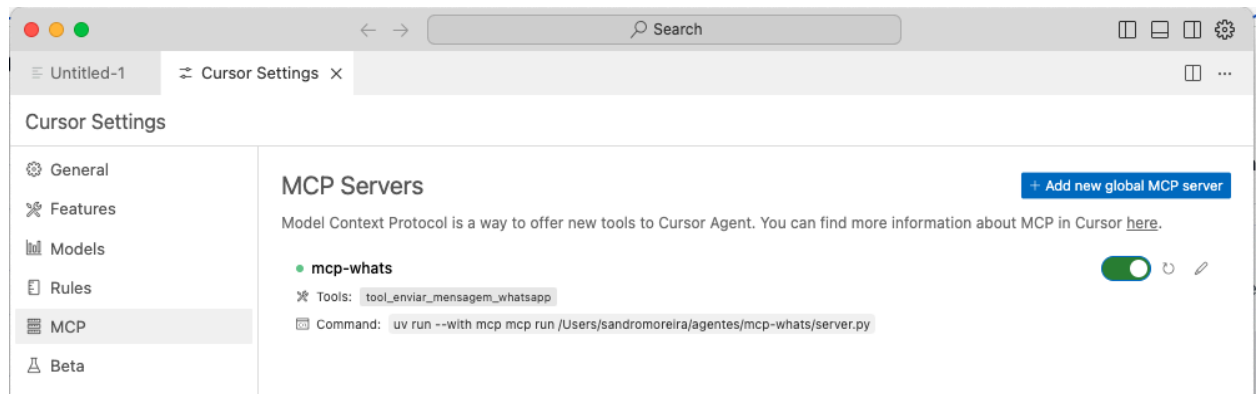
text: "5564992770830@c.us"

}

]

}

Passo 6 - Configuração da IDE Cursor



Users > sandromoreira > .cursor > {} mcp.json > ...

```
1  {
2    "mcpServers": {
3
4      "mcp-whats": {
5        "command": "uv",
6        "args": [
7          "run --with mcp mcp run /Users/sandromoreira/agentes/mcp-whats/server.py"
8        ]
9      }
10   }
11 }
12 }
```

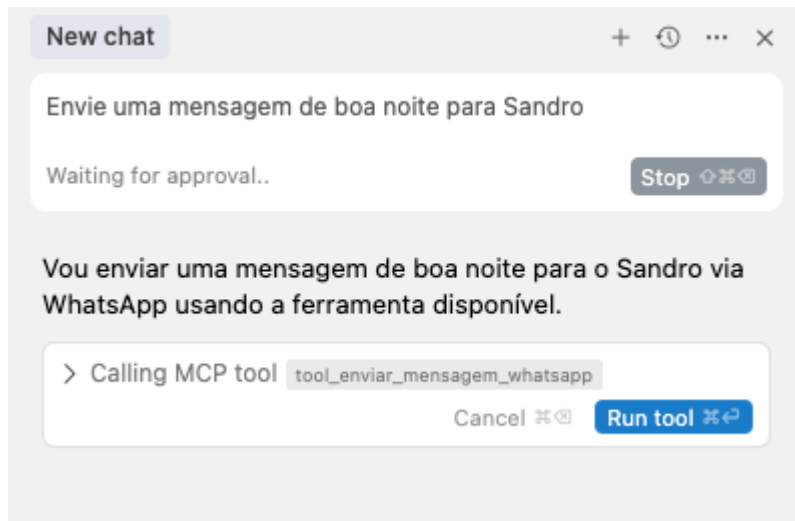
Status do servidor: Online (tool disponibilizada logo abaixo do nome do servidor)

● mcp-whats

✂ Tools: tool_enviar_mensagem_whatsapp

📄 Command: uv run --with mcp mcp run /Users/sandromoreira/agentes/mcp-whats/server.py

Passo 7 - Utilizando ferramenta implementada no servidor MCP



Expandindo mensagem

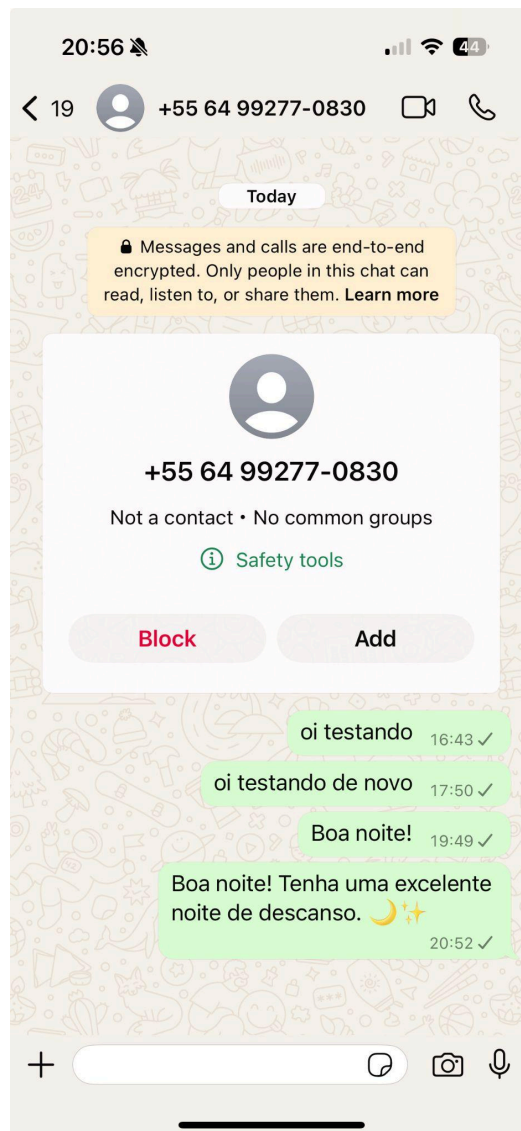


A ferramenta **tool_enviar_mensagem_whatsapp** foi corretamente acionada pelo LLM.

Ao clicar em "Run tool"



Mensagem recebida no aplicativo WhatsApp do usuário



Implementação concluída e testes realizados com sucesso. Códigos disponíveis através do repositório:

<https://github.com/smoreira/mcp-waha>